



OXETECH Academy



SECTI
Secretaria de Estado de
Tecnologia e Inovação



 Softex

REGISTERED BY
AGÊNCIA REGULADORA
DE PROTEÇÃO

CONSUMIDOR FEDERAL
BRASIL
CONSUMIDOR FEDERAL



O OxeTech nasce para promover a capacitação técnica e profissional dos alagoanos, incentivando o empreendedorismo e atraindo investimentos. O público-alvo inclui estudantes do ensino médio e superior, professores e profissionais interessados em tecnologia, oferecendo um espaço moderno para o desenvolvimento de conhecimento.

Quem sou eu?





Sou Luis Pedrosa, líder de qualidade, atuando entre a estratégia e a execução técnica para construir organizações de engenharia e qualidade escaláveis, confiáveis e orientadas a valor. Minha atuação conecta engenharia, qualidade, DevOps e negócio, garantindo que decisões técnicas se traduzam em entregas consistentes e impacto real para o usuário.

Tenho forte background em Quality Engineering, automação em escala e governança de engenharia, liderando times multidisciplinares e definindo estratégias de testes, confiabilidade e entrega contínua. Atuo na evolução de arquiteturas distribuídas e cloud-native, integração de qualidade em pipelines CI/CD, definição de OKRs e métricas de engenharia, além da gestão de riscos técnicos e operacionais.

Cronograma



Módulos	Conteúdo
Módulo 1 — Fundamentos da Engenharia Moderna	Processos, versionamento e code review Gestão de dependências, Documentação viva
Módulo 2 — Clean Code e SOLID	Legibilidade, funções e acoplamento, SOLID aplicado, Refatoração incremental
Módulo 3 — Back -end com Node.js	Factory, Strategy, Observer, Repository, Adapter, Decorator, Anti -patterns comuns

Módulos	Conteúdo
Módulo 4 — Arquiteturas e Escalabilidade	Monólito modular, hexagonal, Microserviços e mensageria, DDD básico
Módulo 5 — Qualidade, Testes e Segurança	Pirâmide de testes e cobertura, Sonar, linters, quality gates, Segurança básica em apps
Módulo 6 — CI/ CD e Operação	Pipelines (build/ test/ deploy), Blue green, canary e rollback, Observabilidade (logs/ metrics/ traces)
Módulo 7 — Projeto Final Integrador	Sistema modular com integração contínua, Qualidade e observabilidade integradas, Defesa técnica

Acordo de aula



Todas as aulas será gravada. Os vídeos ficam disponíveis para os alunos durante todo o período do curso, seja para quem perdeu um encontro ou para quem quiser revisar o conteúdo.

Alunos que acumularem três faltas consecutivas sem justificativa estão sujeitos ao desligamento do curso, conforme as regras do programa.

- **Participação e frequência: 20%**
- **Exercícios práticos: 30% - Apresentação ou Entrega Escrita;**
- **Projeto Final Integrador: 50%**
- **Critério de aprovação: média $\geq 7,0$ e frequência mínima de 75%**

Módulo 1 — Fundamentos da Engenharia Moderna



Engenharia de Software é a área da engenharia dedicada ao desenvolvimento sistemático, disciplinado e mensurável de softwares, garantindo qualidade, eficiência e manutenção contínua. Ela envolve desde a análise de requisitos até testes e atualizações, sendo essencial para criar sistemas confiáveis e escaláveis

Linha do Tempo da Engenharia de Software



Século XIX – Primeiros conceitos

- **Charles Babbage:** projetou a Máquina Analítica, precursora dos computadores.
- **Ada Lovelace:** escreveu o primeiro algoritmo, considerada a primeira programadora.

Décadas de 1950–1960 – Computadores e crise

- **1950s:** surgimento dos transistores e linguagens de alto nível (Fortran, COBOL).
- **1968:** Conferência da **NATO** cunha o termo *Engenharia de Software* em resposta à “crise do software” (falhas frequentes, custos imprevisíveis).
- **Margaret Hamilton:** populariza práticas formais e disciplinares no desenvolvimento.

Década de 1970 – Estruturação	• Unix reescrito em C (1973): marco da portabilidade. • Modelo Cascata (Waterfall): primeira metodologia estruturada de ciclo de vida. • Microcomputadores (1975): democratizam o acesso e ampliam a demanda por software.
Década de 1980 – Consolidação	• Linguagens como Pascal e Smalltalk: introduzem paradigmas estruturados e orientados a objetos. • Ferramentas CASE: auxiliam no design e documentação. • No Silver Bullet (1987): artigo de Fred Brooks sobre limites da produtividade em software.

Década de 1990 – Internet e código aberto

- **Ascensão da Web:** softwares passam a ser distribuídos globalmente.
- **Linux e movimento open source:** fortalecem colaboração e inovação.
- **Metodologias incrementais** começam a ganhar espaço.

Engenharia de Software Moderna



Década de 2010 – Mobilidade e nuvem	• Aplicativos móveis dominam o mercado. • Computação em nuvem transforma distribuição e escalabilidade. • DevOps surge como integração entre desenvolvimento e operações.
Década de 2020 – Inteligência Artificial e sustentabilidade	• IA aplicada ao desenvolvimento: geração de código, testes automatizados e análise preditiva. • Engenharia de software sustentável: foco em eficiência energética e impacto ambiental.

Ciclo e Processo de Desenvolvimento de Software

O **Software Development Life Cycle (SDLC)** é um **processo estruturado** que descreve todas as etapas necessárias para criar um software — desde a ideia inicial até a manutenção após o lançamento. Ele funciona como um **guia** para equipes de desenvolvimento, garantindo que o produto final seja **útil, confiável e de qualidade**.

Etapas principais do SDLC



Planejamento	• Definir objetivos, custos, prazos e recursos. • Exemplo: decidir se o software será um aplicativo de celular ou um sistema web.
Análise de requisitos	• Levantar o que o usuário precisa. • Exemplo: o app deve permitir login, cadastro e envio de mensagens.
Design (projeto)	• Criar a arquitetura e o desenho do sistema. • Exemplo: definir telas, banco de dados e fluxos de navegação.

Implementação (codificação)	• Programar o software usando linguagens como Java, Python ou JavaScript.
Testes	• Verificar se tudo funciona corretamente e sem erros. • Exemplo: testar se o login aceita apenas senhas válidas.
Implantação (deploy) e Manutenção	• Colocar o software em uso real, disponível para os usuários. • Corrigir problemas, atualizar funções e melhorar desempenho.

Por que o SDLC é importante?



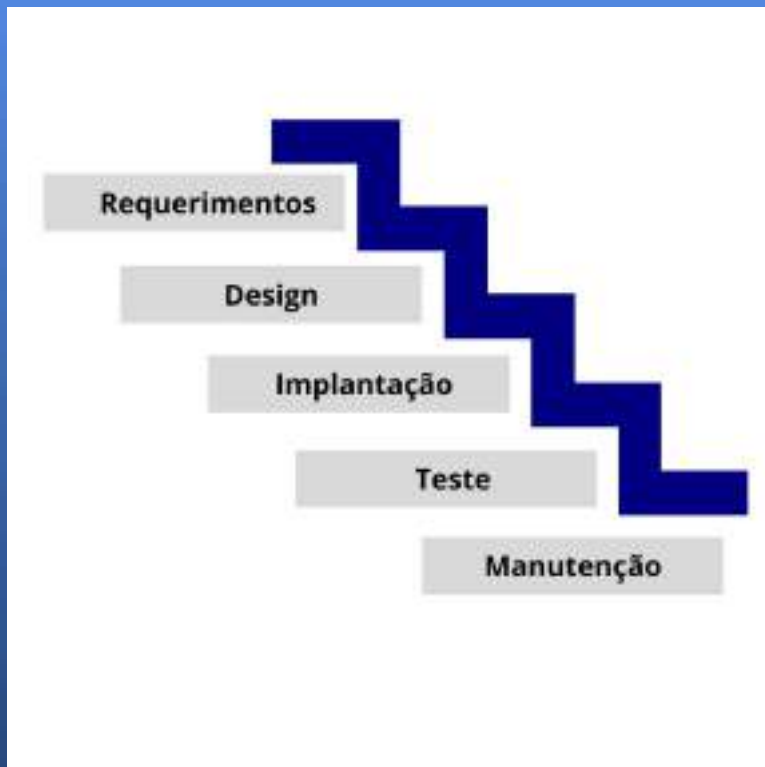
Organização: evita improvisos e falhas graves.

Qualidade: garante que o software atenda às necessidades do usuário.

Previsibilidade: ajuda a controlar tempo e custos.

Evolução: permite que o software seja atualizado ao longo do tempo.

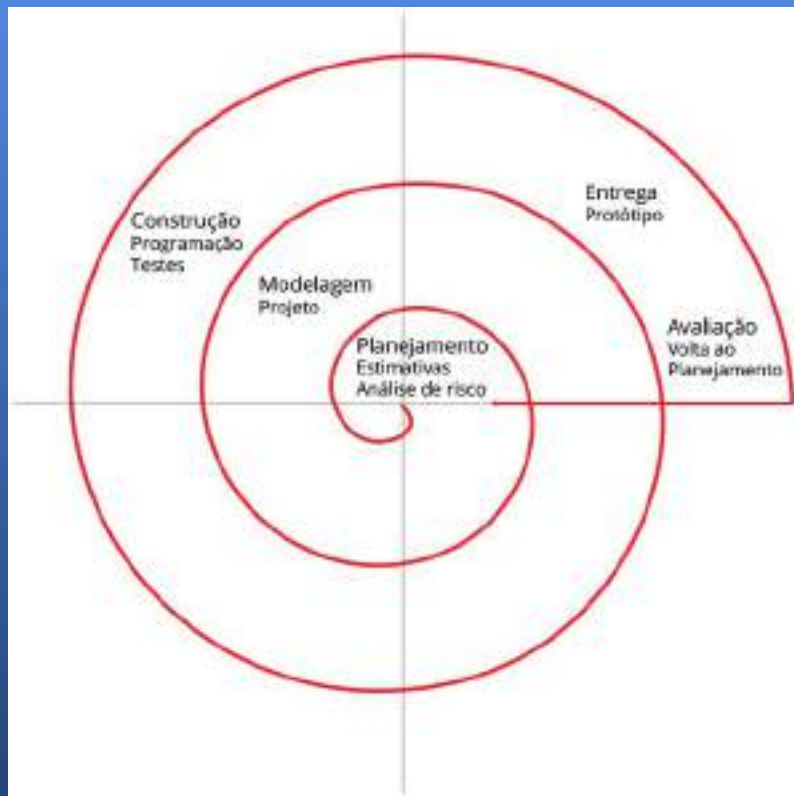
Modelos de SDLC = Processo de Desenvolvimento de Software



O **modelo Cascata** é um dos primeiros e mais tradicionais modelos de ciclo de desenvolvimento de software. Ele foi criado nos anos 1970 e é chamado assim porque as etapas acontecem de forma **sequencial**, como uma “cascata”: cada fase precisa ser concluída antes que a próxima comece.

Característica

- **Sequencial e rígido:** não permite voltar facilmente a etapas anteriores.
- **Documentação extensa:** cada fase gera documentos formais.
- **Previsibilidade:** ideal para projetos bem definidos e estáveis.



O **modelo de desenvolvimento Espiral** é uma forma de organizar o ciclo de vida do software que surgiu em 1986, proposta por Barry Boehm. Ele combina elementos do **modelo Cascata** (sequência de etapas) com a ideia de **iterações repetidas**, permitindo que o projeto seja refinado em ciclos sucessivos.

Característica

- **Iterativo:** o software é desenvolvido em partes, com revisões constantes.
- **Foco em riscos:** cada ciclo inclui análise e mitigação de riscos.
- **Flexível:** permite mudanças de requisitos ao longo do projeto.
- **Prototipagem:** uso de protótipos para validar ideias antes da implementação final.

Processo de Desenvolvimento Agil



O **modelo de desenvolvimento Ágil** é uma abordagem moderna para criar software que surgiu oficialmente em 2001, com o **Manifesto Ágil**. Ele foi criado como resposta às limitações dos modelos tradicionais (como o Cascata), que eram muito rígidos e demorados.

O Ágil é baseado em **ciclos curtos de trabalho** chamados *iterações* ou *sprints*. Em cada ciclo, a equipe entrega uma parte funcional do software, recebe feedback dos usuários e ajusta o que for necessário. A ideia central é: **adaptar-se rapidamente às mudanças e entregar valor contínuo ao cliente.**

Princípios do Manifesto Ágil

Indivíduos e interações mais importantes que processos e ferramentas.

Software funcionando mais importante que documentação extensa.

Colaboração com o cliente mais importante que contratos rígidos.

Responder a mudanças mais importante que seguir um plano fixo.

Como funciona na prática

Sprints: ciclos curtos (geralmente 2 a 4 semanas).

Reuniões rápidas (daily stand-ups): para alinhar o progresso.

Backlog: lista de tarefas priorizadas.

Feedback contínuo: o cliente participa ativamente do processo.

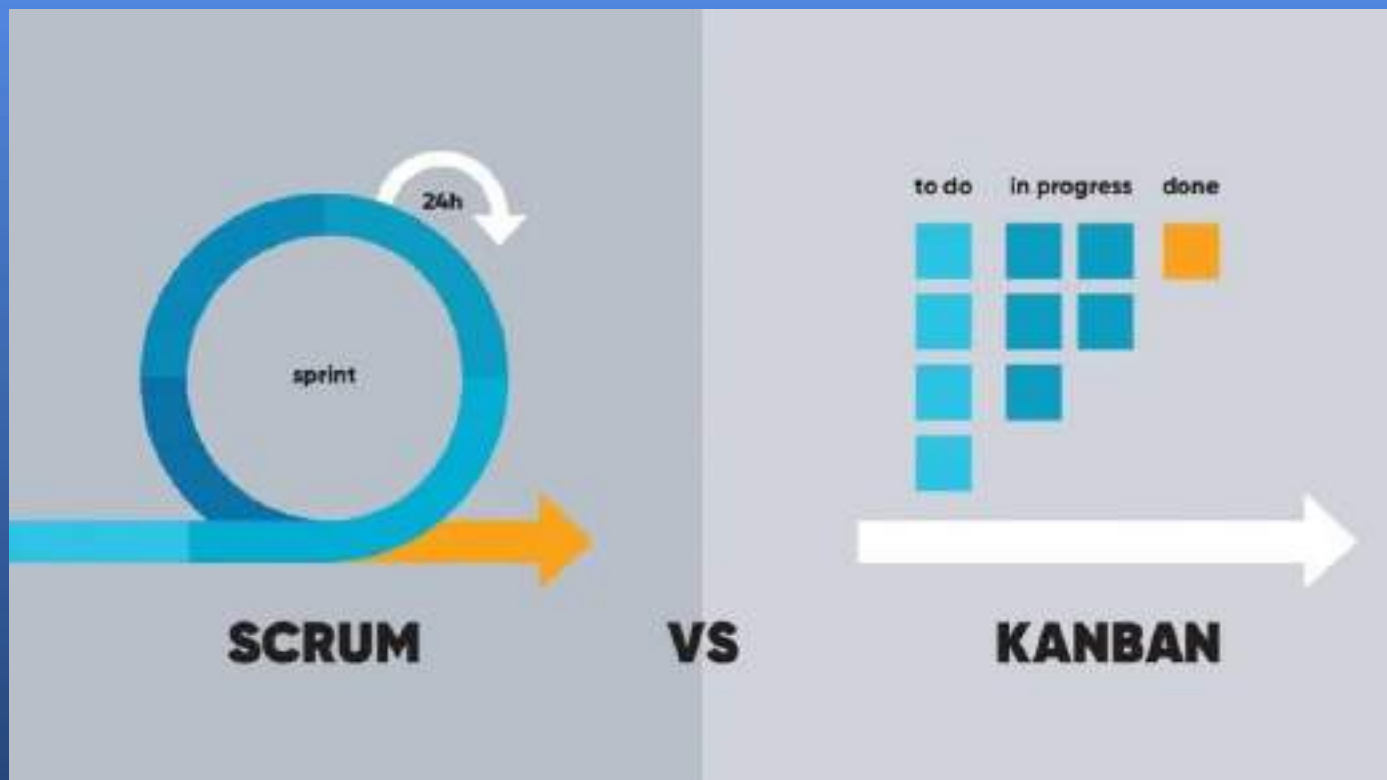
Entrega incremental: o software vai sendo liberado em partes utilizáveis.

Metodologias Ágeis



Vamos praticar?





Documentação viva

Documentação Viva é uma forma de manter a documentação de software sempre atualizada e útil, integrada diretamente ao processo de desenvolvimento. Diferente da documentação tradicional (que muitas vezes fica desatualizada), ela é **gerada e validada automaticamente** junto com o código e os testes.

Gerenciamento de Versão

Basicamente, os arquivos do projeto ficam armazenados em um repositório (um servidor em outras palavras) e o histórico de suas versões é salvo nele. Os desenvolvedores podem acessar e resgatar a ultima versão disponível e fazer uma cópia local, na qual poderão trabalhar em cima dela e continuar o processo de desenvolvimento. A cada alteração feita, é possível enviar novamente ao servidor e atualizar a sua versão a partir outras feitas pelos demais desenvolvedores.

Atualmente, os sistemas de controle de versão são classificados em dois tipos:

Centralizados e distribuídos.



O **centralizado** trabalha apenas com um servidor central e diversas áreas de trabalho, baseados na arquitetura cliente-servidor. Por ser centralizado, as áreas de trabalho precisam primeiro passar pelo servidor para poderem comunicar-se. Essa versão atende muito bem a maioria das equipes de desenvolvimento que não sejam enormes e trabalhem em uma rede local, além de não ter problemas de velocidade para enviar e receber os dados e ter um bom tempo de resposta do servidor. Um dos principais sistemas com o tipo de controle de versão centralizado é o Subversion.

O **distribuído** vai mais além. Ele é recomendado para equipes com muitos desenvolvedores e que se encontram em diferentes filiais. Esta versão funciona da seguinte maneira: cada área de trabalho tem seu próprio “servidor”, ou seja, as operações de check-in e check-out são feitas na própria máquina. Porém diferentemente do centralizado, as áreas de trabalho podem comunicar-se entre si, recomenda-se usar um servidor como centro do envio dos arquivos para centralizar o fluxo e evitar ramificações do projeto e a perda do controle sobre o mesmo.

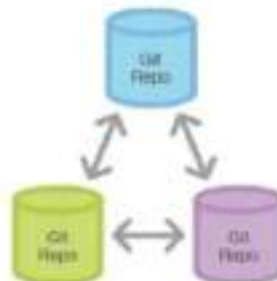
GIT x SVN

Central-Repo-to-Working-Copy
Collaboration

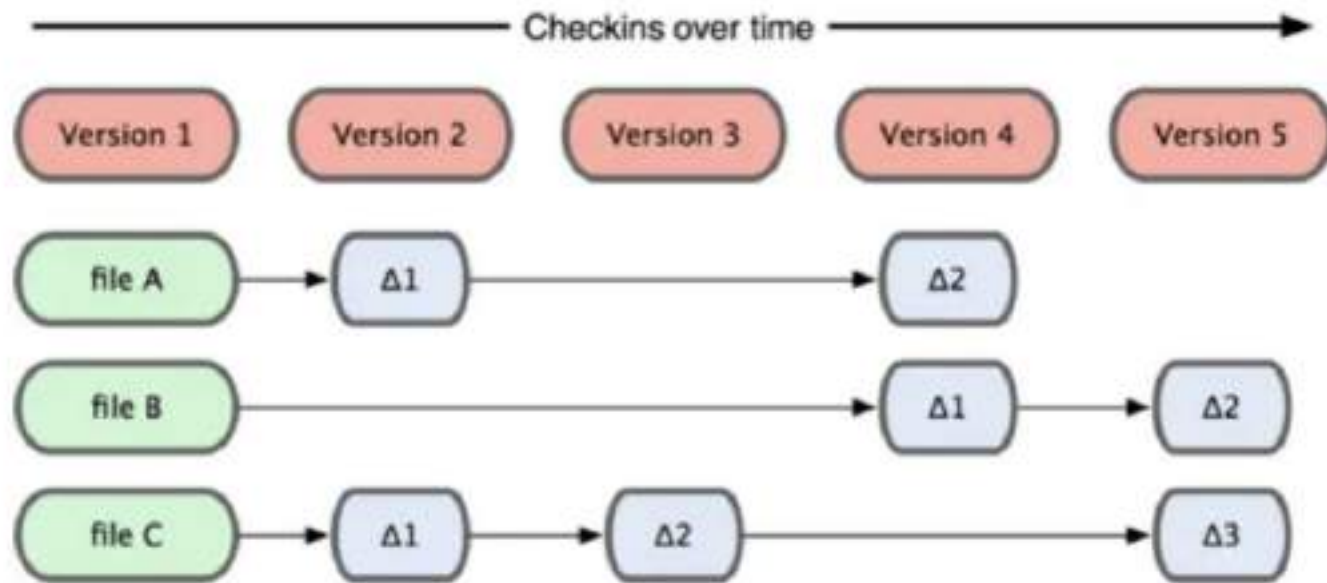


SVN - Centralizado

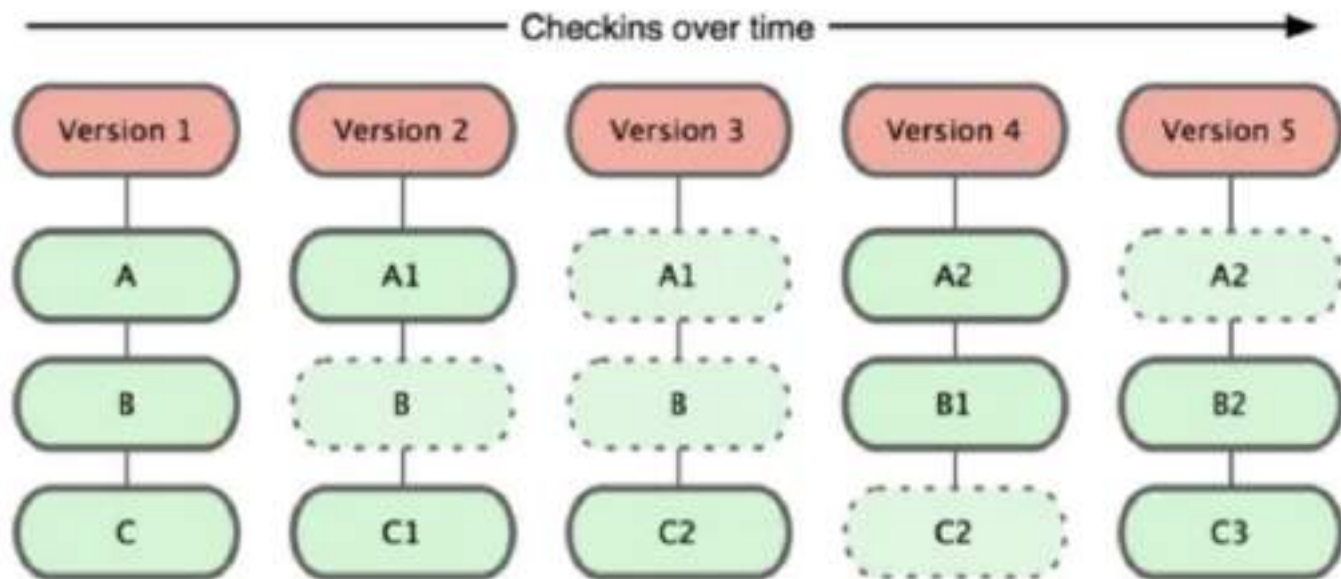
Repo-to-Repo
Collaboration



Git - Distribuído



Controle de mudanças do SVN



Controle de mudanças no Git



Git vs GitHub

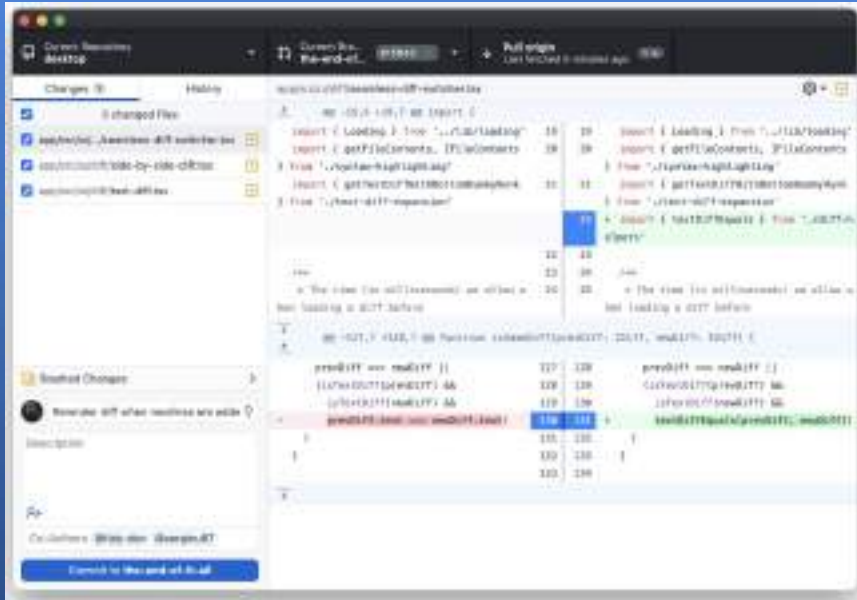


Git: é o sistema de controle de versão. Ele roda no seu computador e guarda o histórico de alterações.

GitHub: é um site que hospeda projetos que usam Git. Ele permite compartilhar código com outras pessoas, colaborar em equipe e guardar tudo na nuvem.



<https://git-scm.com/install/mac>



<https://github.com/desktop/desktop>

Conceitos básicos do GitHub



Repositório (Repository)	• É como uma pasta online onde o projeto fica guardado. • Contém todos os arquivos, histórico de alterações e configurações. • Pode ser público (qualquer pessoa vê) ou privado (apenas quem você autorizar).
Commit	• É um registro de alteração feito no código. • Cada commit guarda o que mudou e uma mensagem explicando. • Funciona como um “salvar com histórico”.

Branch	• É uma ramificação do projeto. • Permite criar novas funcionalidades ou testar ideias sem mexer no código principal. • Depois, pode ser unida ao projeto principal.
Merge	• É o ato de juntar uma branch com o código principal. • Une as alterações feitas em paralelo
Pull Request (PR)	• É um pedido para revisar e aprovar mudanças antes de juntá-las ao projeto principal. • Muito usado em trabalhos em equipe e projetos open source.

Fork	• É uma cópia de um repositório de outra pessoa para o seu perfil. • Permite modificar e propor melhorias sem alterar o original.
Clone	• É baixar o repositório do GitHub para o seu computador. • Assim você pode trabalhar localmente e depois enviar as mudanças de volta.

Criar conta no GitHub – gratuita e rápida.

Instalar o Git no computador.

Criar um repositório no GitHub para seu projeto.

Clonar o repositório para o seu computador (`git clone`).

Fazer alterações no código e salvar com `git commit`.

Enviar para o GitHub com `git push`.

Colaborar com outros usando branches e pull requests.

GitFlow: Modelo Estruturado

O Git Flow é um fluxo de trabalho (workflow) para Git criado para organizar melhor o versionamento de projetos, também chamado de gitflow. Muitos procuram no Google por 'gitflow o que é' ou 'o que é gitflow': trata-se de uma metodologia para estruturar diferentes tipos de branch, como develop, master, release, feature e hotfix, trazendo clareza na hora de contribuir em equipes.

Quando utilizar o Git Flow?

O Git Flow é indicado para projetos que seguem versionamento semântico (**semantic versioning**) ou que precisam manter e dar suporte a **múltiplas versões do software ao mesmo tempo**.

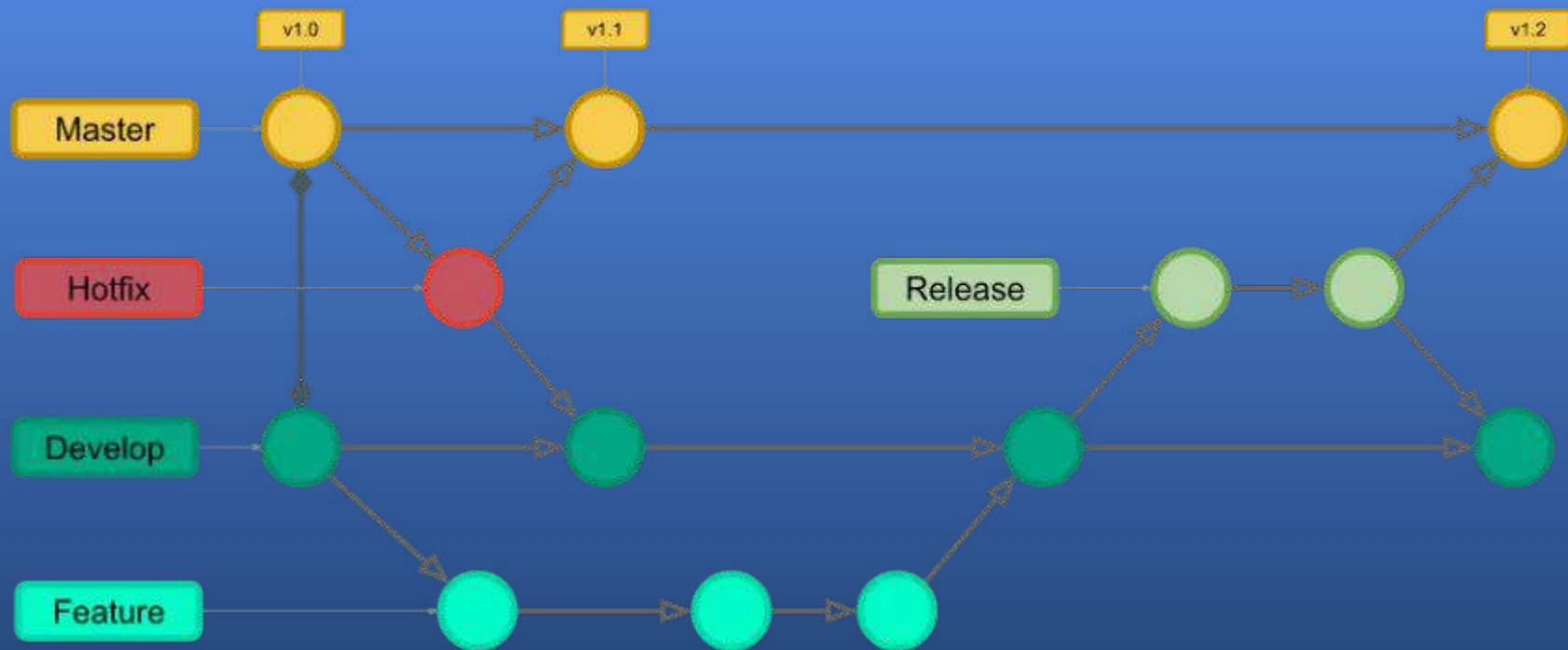
A utilização do Git Flow também é indicada para **equipes grandes**, onde muitas pessoas **contribuem simultaneamente em um repositório**, ou para **projetos com ciclos de entrega planejados e agendados**.

Como funciona o Git Flow?

O Git Flow opera com duas branches principais (**Develop e Master**), que são permanentes, e três branches de suporte (**Feature, Release e Hotfix**), que são temporárias e existem apenas até serem mescladas (merge) nas branches principais.

Então, ao invés de uma única **branch Master**, esse fluxo de trabalho utiliza duas branches principais para registrar o histórico do projeto. A branch Master armazena o histórico do lançamento oficial, e a branch Develop serve como uma ramificação de integração para recursos.

É ideal que todos os commits na **branch Master sejam marcados com um número de versão**. Na imagem abaixo, vemos como é a estrutura do fluxo do Git Flow:



GitHub Flow: Simplicidade

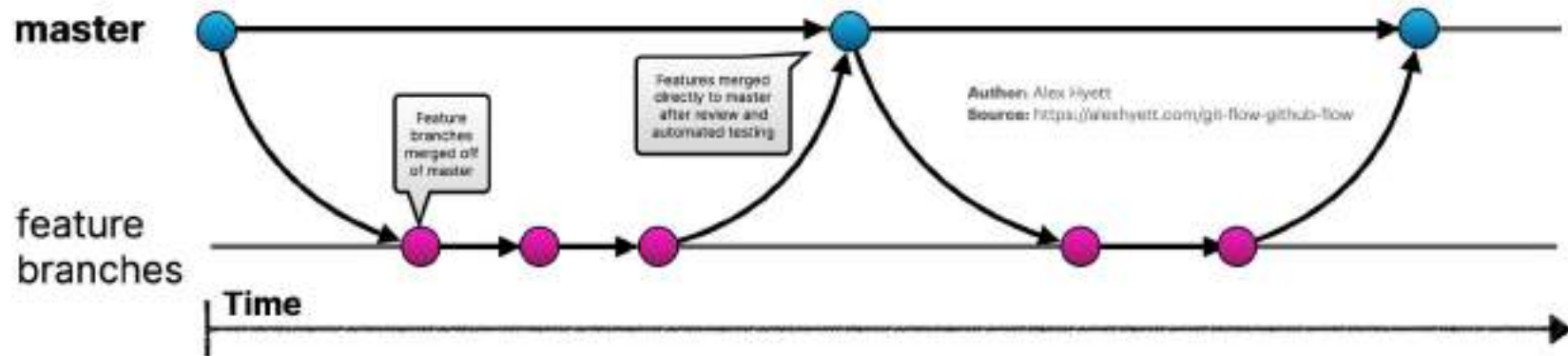


O GitHub Flow é uma metodologia de desenvolvimento que simplifica a colaboração e o controle de versão, promovendo um fluxo de trabalho ágil e eficiente.

Através do uso estratégico de branches e pull requests, as equipes podem gerenciar o desenvolvimento de novas funcionalidades, correções de bugs e melhorias de forma organizada e previsível.

- **Branch Principal (main):** Serve como a versão estável do projeto, refletindo o estado de produção.
- **Feature Branches:** Para cada nova funcionalidade ou correção, cria-se uma branch específica a partir da main.
- **Commits:** As alterações são registradas através de commits claros e descritivos na branch correspondente.
- **Pull Requests:** Após concluir o desenvolvimento ou a correção, abre-se um pull request para revisão e discussão.
- **Revisão e Merge:** Após a aprovação, a branch é mesclada de volta na main, garantindo que apenas código validado seja integrado.
- **Implantação:** A main é implantada em produção, garantindo que as últimas alterações aprovadas estejam disponíveis.

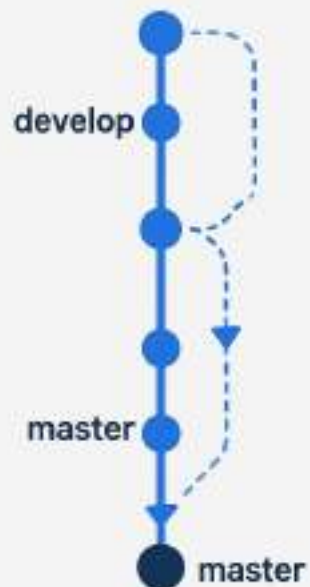
GitHub Flow



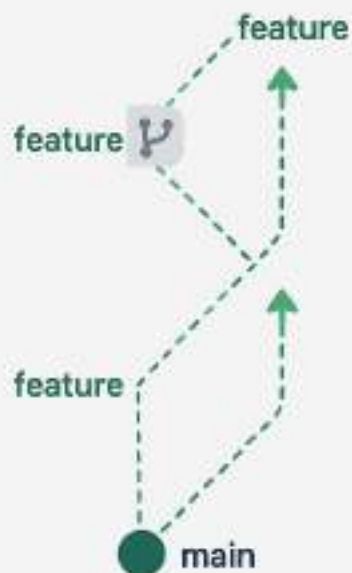
Git Flow vs GitHub Flow



Gitflow



GitHub Flow



Trunk-Based Development



Um modelo de ramificação de controle de versão, onde os desenvolvedores colaboram no código em uma única ramificação chamada 'trunk'* e resistem a qualquer pressão para criar outras ramificações de desenvolvimento de longa duração, empregando técnicas documentadas. Dessa forma, eles evitam o inferno das mesclagens, não quebram a compilação e vivem felizes para sempre.

* *Principal* para a comunidade Git desde 2020 (master com conotações desagradáveis antes disso)



Comparação de Estratégias em Cenários Reais



Startup

TAMANHO

1 pessoa

ESTRATÉGIA

GitHub Flow

RAZÃO

Simplicidade, deploys frequentes

DEPLOY

1-2x por dia

EXEMPLO

Aplicação web SaaS

Empresa Média

TAMANHO

50 pessoas

ESTRATÉGIA

GitHub Flow + Feature Flags

RAZÃO

Escalabilidade, controle de features

DEPLOY

2-4x por dia

EXEMPLO

Plataforma com múltiplos módulos

Empresa Grande

TAMANHO

100+ pessoas

ESTRATÉGIA

Trunk-Based Development

RAZÃO

Escalabilidade extrema, reduz conflitos

DEPLOY

10-50x por dia

EXEMPLO

Google, Netflix, Microsoft

Open Source

TAMANHO

Comunidade

ESTRATÉGIA

GitFlow

RAZÃO

Releases planejados, múltiplas versões

RELEASE

1-4x por mês

EXEMPLO

Linux, Apache, Node.js

Versionamento Semântico



O desenvolvimento iterativo e incremental exige muito com relação à cooperação, organização e segurança no desenvolvimento de software. Adotar a regra de versionamento semântico significa etiquetar conjuntos de mudanças (pull-requests), de modo análogo a gavetas de um arquivo. É possível otimizar, hoje mesmo, o desenvolvimento de suas aplicações com *semantic versioning 2.0*:

- Centralização de trabalhos;
- Segurança para integrações;
- Organização para trabalho em equipe;
- Etiquetagem/backup dos trabalhos.

Como funciona o versionamento semântico



1.0.0



MAJOR

MINOR

PATCH

MAJOR (Maior)

O primeiro dígito informa a versão de compatibilidade e é alterado caso o software ou biblioteca sofra mudanças que a torne incompatível com outras versões. São as chamadas *breaking changes*, atualizações que possuem o potencial de “quebrar” códigos que utilizam versões anteriores.

Exemplo: você está usando uma função de uma biblioteca X, porém foi lançada uma nova versão da biblioteca onde essa função tem outro nome ou outra assinatura. Se tentarmos executar o código usando a versão nova da biblioteca, a função não executará corretamente.

Versão 1.0.0 → Agora é 2.0.0

MINOR (Menor)

O segundo dígito informa a versão da funcionalidade, onde uma nova função ou melhoria substancial é adicionada e não há problemas de incompatibilidade com outras versões.

Exemplo: A biblioteca que você costuma usar tem agora uma nova funcionalidade e é compatível com outras versões, necessita apenas de atualização local.

Versão 2.0.0 → Agora é 2.1.0

PATCH (Correção)

O terceiro dígito informa a versão da correção de bugs, melhorias de desempenho ou alterações similares que não alteram as funcionalidades atuais e nem introduzem novas.

Exemplo: A biblioteca que você costuma usar tem um bug que gera uma vulnerabilidade no código. Esse bug foi corrigido em uma nova versão.

Versão 2.1.0 → Agora é 2.1.1

 EmersonLaranja / versionamento-semantic Public

 Pin  Unwatch 1  Fork 0  Star 0

 Code  Issues  Pull requests  Actions  Projects  Wiki  Security  Insights  Settings

 main  1 branch  0 tags

 Go to file  Add file  Code

 EmersonLaranja começando desenvolvimento titoxw@ · 32 seconds ago · 1 commit

 index.js	começando desenvolvimento	32 seconds ago
 package.json	começando desenvolvimento	32 seconds ago

Help people interested in this repository understand your project by adding a README. 

About

No description, website, or topics provided.

☆ 0 stars

👁 1 watching

🍴 0 forks

Releases

No releases published

[Create a new release](#)

EmersonLaranja / [versionamento-semantic](#) Public

Pin

Unwatch

Fork 0

Star 0

[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#) [Security](#) [Insights](#) [Settings](#)

main

1 branch

1 tag

Go to file

Add file

Code

About



No description, website, or topics provided.

0 stars

1 watching

0 forks



EmersonLaranja começando desenvolvimento

599 lines · 12 minutes ago · 1 commit



index.js

começando desenvolvimento

12 minutes ago



package.json

começando desenvolvimento

12 minutes ago

Help people interested in this repository understand your project by adding a README.

Add a README

Releases

1 tag

Create a new release

EmersonLaranja / versionamento-semantico Public

 Pin

<> Code

Issues

 Pull requests

 Actions

 Projects

 Wiki

 Security

 Insights

 Settings

Releases

Tags

 Tags

0.1.0



começando desenvolvimento do projeto



6 minutes ago



399cea0



zip



tar.gz

 EmersonLaranja / **versionamento-semantic** Public Pin Unpin Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Releases

Tags



There aren't any releases here

You can create a release to package software, along with release notes and links to binary files, for other people to use. Learn more about releases in [our docs](#).

[Create a new release](#)

 EmersonLaranja / versionamento-semantic Public

 Pin  Unwatch 1  Fork 0

 Code  Issues  Pull requests  Actions  Projects  Wiki  Security  Insights  Settings

Releases

Tags

Draft a new release

Find a release

28 seconds ago
 EmersonLaranja
v 0.1.0
40 399ceab
Compare

2022-11-24, versão 0.1.0, @EmersonLaranja Preview

Criei a inicialização de um servidor com o Node.js

Assets

Source code (zip) 25 minutes ago

Source code (tar.gz) 25 minutes ago

Cases Reais: Google, Netflix e Microsoft



Google

ESTRATÉGIA

Trunk-Based Development

ESCALA

25.000+ engenheiros
2 bilhões de linhas de código

PROCESSO

- Todos commits para trunk
- Code review obrigatório
- Feature flags para controle

RESULTADO

Deploy múltiplas vezes/dia, zero merge conflicts

LIÇÃO

Automação e testes são críticos para escala

Netflix

ESTRATÉGIA

Git-hub-Flow + Canary

ESCALA

Centenas de microserviços
4.000+ deploys/dia

PROCESSO

- Feature branches com PRs
- Canary: 5% → 25% → 100%
- Monitoramento com Spinnaker

RESULTADO

Zero downtime, deploy seguro

LIÇÃO

Canary reduz risco de falhas

Microsoft

ESTRATÉGIA

Trunk-Based + Feature Flags

ESCALA

Windows, Office, Azure
Múltiplos produtos

PROCESSO

- Commits diários para trunk
- Feature toggles para ativar
- A/B testing em produção

RESULTADO

Atualizações rápidas e seguras

LIÇÃO

Feature flags desacopla deployment

Vamos praticar?



Code Review

Code Review é o processo de revisar o código escrito por outro desenvolvedor antes que ele seja integrado ao projeto principal. O objetivo é garantir **qualidade, legibilidade e segurança** do software, além de promover aprendizado entre a equipe.

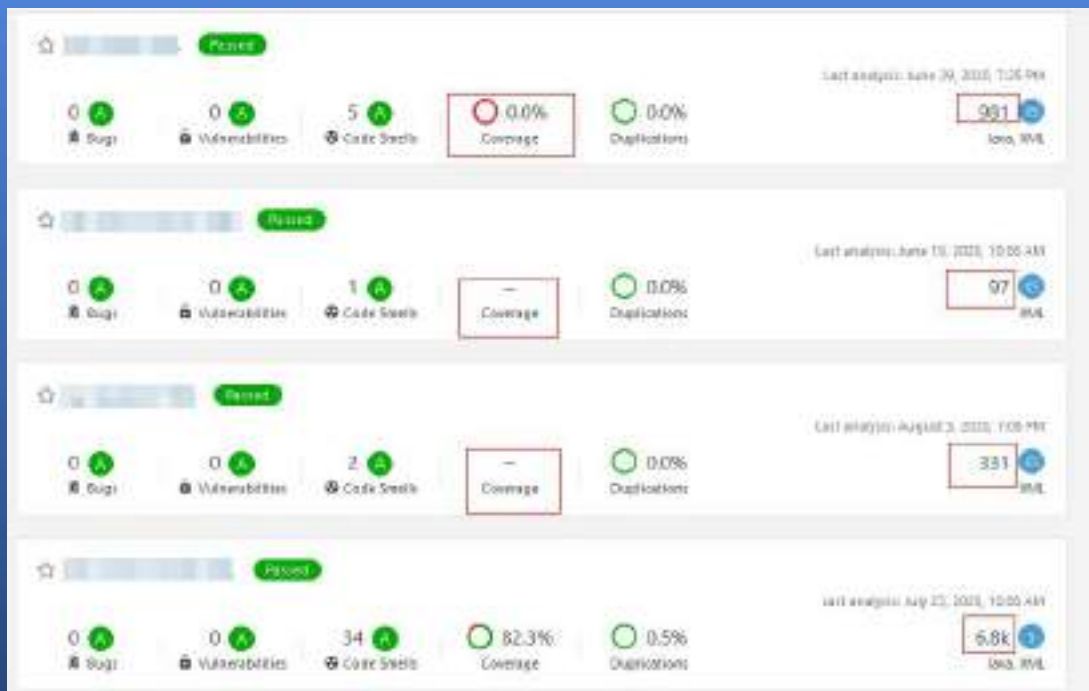
Onde entra o Code Review no SDLC



Requested	To Do	Coding	Code Review	Testing	Deployment	Done
Messaging on Android Application	App Z Integration	Real-time analytics	Drag-and-drop editor for reports	Switching to SQLite	Autosave feature	Responsive menu
Tagging function						Homepage animations
						Bugged graph visualization

Análise Estática de Código





JACOCO

Java Code Coverage

org.jacoco.ant

https://www.eclerema.org/jacoco/trunk/coverage/org.jacoco.ant/org.jacoco.ant/index.source.html

JaCoCo > org.jacoco.ant > org.jacoco.ant

Classes Reports

org.jacoco.ant

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cov.	Missed	Lines	Missed	Methods	Missed	Classes
DummyTask.java		92%		100%	2	16	4	43	2	12	0	2
AbstractCoverageTask.java		92%		100%	0	22	3	49	0	20	0	1
CoverageTask.java		98%		100%	1	17	3	49	1	10	0	3
Benchmark.java		100%		98%	1	74	0	186	0	47	0	8
InstrumentTask.java		100%		100%	0	9	0	35	0	8	0	1
MergeTask.java		100%		100%	0	9	0	32	0	8	0	1
AntResourceLocator.java		100%		100%	0	6	0	17	0	4	0	1
AgentTask.java		100%		100%	0	6	0	8	0	3	0	1
AntFileLocator.java		100%		100%	0	4	0	9	0	3	0	1
Total	23 of 1,737	98%	1 of 104	99%	4	163	8	428	3	111	0	19

JaCoCo Coverage Report for: org.jacoco:3.3.0-20200910.1

Generated with JaCoCo 3.3.0-20200910.1



CodeRabbit

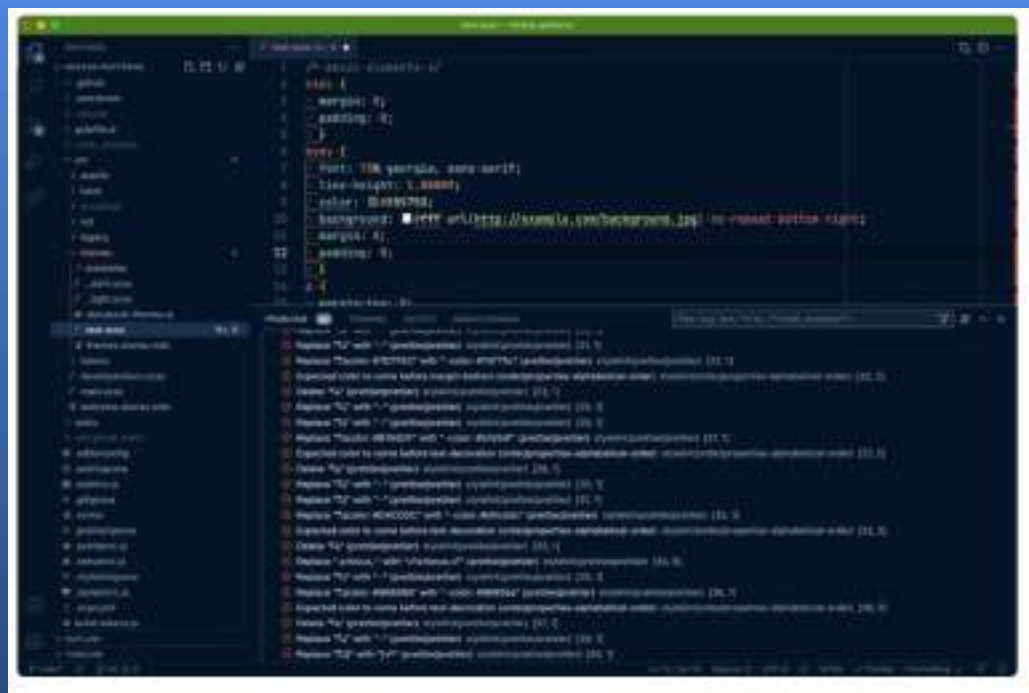
File Explorer | **find_task.py**

```
def get_task_model_from_gql_or_tasktask_gql(task_process_instance_id: int) -> TaskModel:
    task_model: TaskModel | None = None
    TaskModel.query
    After_get_gql(task_gql, process_instance_id, process_instance_id)
    if task_model is None:
        raise ValueError(
            "Task not found with gql (task_gql)"
        )
    return task_model
```

CodeRabbit - 10 min ago

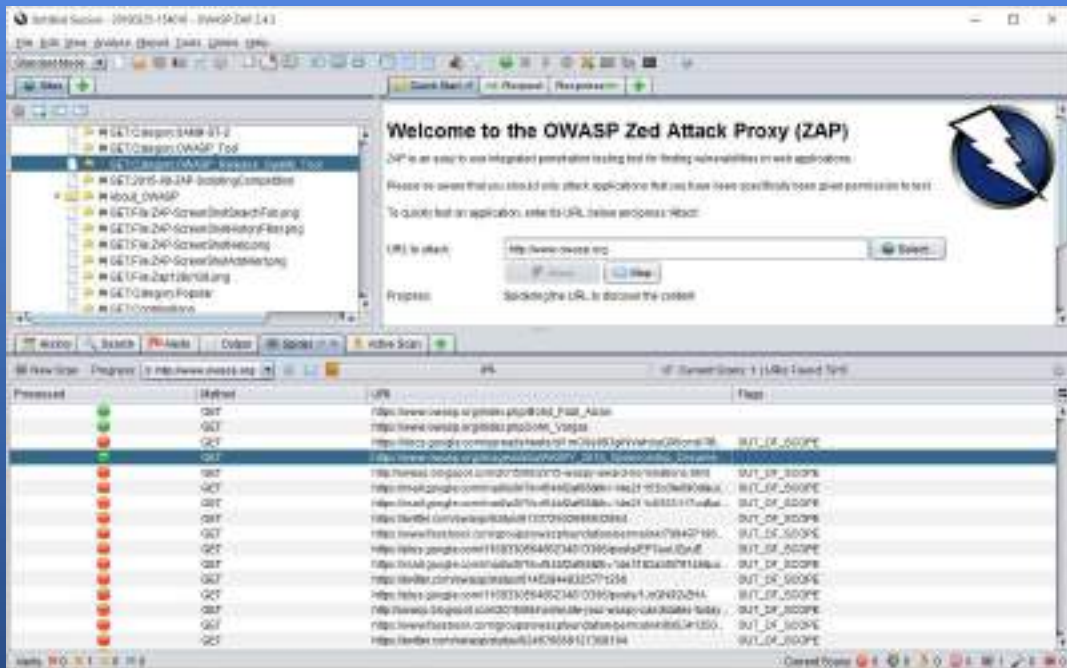
Status code 400 may not be appropriate for all use cases where this function is called. It is suggested to use 404 instead, as it is more suitable for not found errors.

```
- status_code=400,
+ status_code=404,
```

Análise de Vulnerabilidade





Codew Review com Github



Criar uma branch	● Desenvolvedor cria uma branch e faz alterações no código.
Commit e Pull Request (PR)	● Ele envia suas mudanças para revisão.
Revisores analisam o código	● Verificam clareza, eficiência, segurança e aderência às regras do projeto.
Feedback	● Podem aprovar, pedir ajustes ou sugerir melhorias.
Merge	● Após aprovação, o código é integrado ao repositório principal.

Revisar pequenas mudanças



Criar uma branch	● Desenvolvedor cria uma branch e faz alterações no código.
Commit e Pull Request (PR)	● Ele envia suas mudanças para revisão.
Revisores analisam o código	● Verificam clareza, eficiência, segurança e aderência às regras do projeto.
Feedback	● Podem aprovar, pedir ajustes ou sugerir melhorias.
Merge	● Após aprovação, o código é integrado ao repositório principal.

Melhores práticas de Code Review no SDLC



Revisar pequenas mudanças



Revisar pequenas mudanças	• Evite Pull Requests muito grandes. • Revisões menores são mais rápidas e reduzem a chance de erros passarem despercebidos.
Usar checklists	• Criar uma lista de pontos a verificar: legibilidade, testes, segurança, performance, padrões de estilo. • Isso garante consistência e evita esquecimentos.
Automatizar o que for possível	• Ferramentas de lint, testes automatizados e CI/CD devem rodar antes da revisão. • Assim, o revisor foca em lógica e arquitetura, não em erros triviais.

Revisar pequenas mudanças

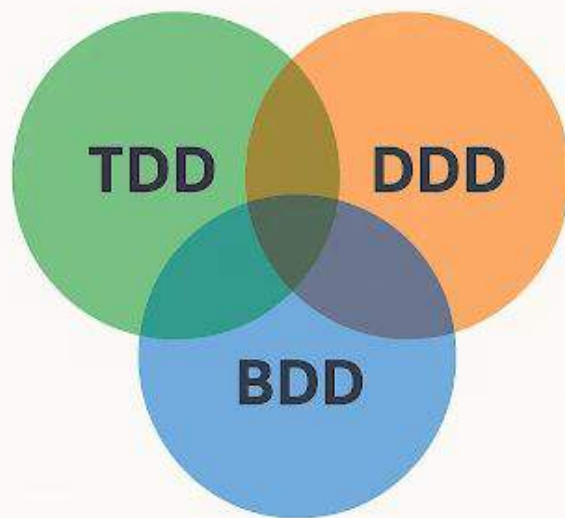


Dar feedback construtivo	• Comentários devem ser claros, respeitosos e sugerir soluções. • O objetivo é melhorar o código, não criticar o desenvolvedor.
Promover aprendizado	• Revisores mais experientes podem explicar boas práticas. • Isso transforma o Code Review em uma oportunidade de crescimento para toda a equipe.
Definir critérios de aprovação	• Estabelecer regras: quantos revisores precisam aprovar, quais tipos de mudanças exigem mais atenção. • Isso dá clareza e evita conflitos.

Documentação viva

Documentação Viva é uma forma de manter a documentação de software sempre atualizada e útil, integrada diretamente ao processo de desenvolvimento. Diferente da documentação tradicional (que muitas vezes fica desatualizada), ela é **gerada e validada automaticamente** junto com o código e os testes.

Linguagem Ubíqua: é uma linguagem estruturada em torno do modelo de domínio e usada por todos os membros da equipe para conectar todas as suas atividades com o software. Numa equipe de desenvolvedores são: os jargões técnicas, terminologias das discussões do dia-a-dia ou uma linguagem incomum para pessoas de outros departamentos.



Vantagens em usar BDD



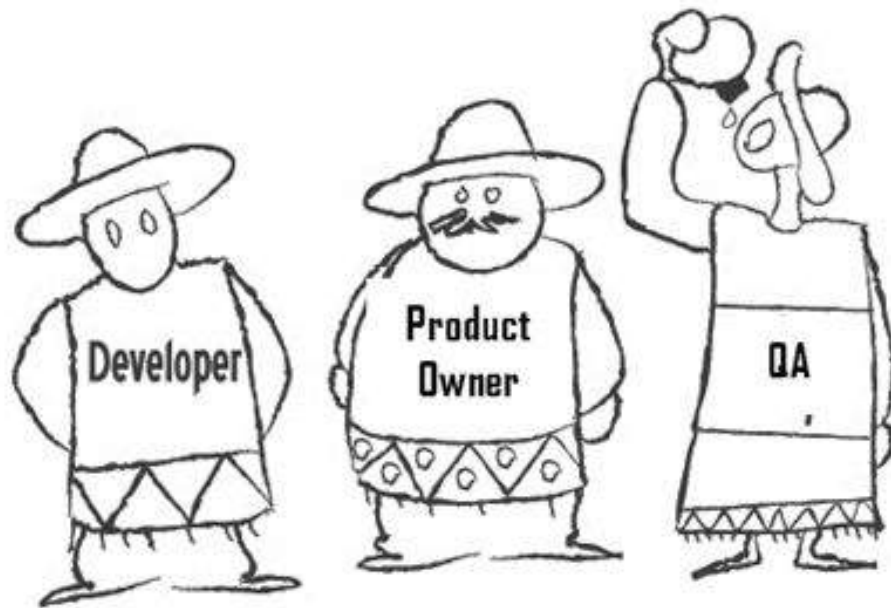
- **Comunicação entre equipes:** na maioria das empresas de desenvolvimento de software é difícil fazer com que desenvolvedores e testadores trabalhem em conjunto para atingir um objetivo. **BDD possibilita esta integração porque os testadores podem escrever os cenários de testes para os desenvolvedores implementarem;**
- **Compartilhamento de conhecimento:** com desenvolvedores e testadores trabalhando juntos, ao longo do tempo, um irá transferir o seu conhecimento para o outro, criando assim uma equipe multifuncional;
- **Documentação dinâmica:** algumas equipes ágeis afirmam que não documentam o sistema porque a manutenção destes artefatos é custosa. Usando os frameworks de BDD estes artefatos são gerados dinamicamente sem nenhum esforço adicional. Alguns, inclusive, geram relatórios em formato HTML, o que irá facilitar uma consulta posterior;

- **Comunicação entre equipes:** na maioria das empresas de desenvolvimento de software é difícil fazer com que desenvolvedores e testadores trabalhem em conjunto para atingir um objetivo. **BDD possibilita esta integração porque os testadores podem escrever os cenários de testes para os desenvolvedores implementarem;**
- **Compartilhamento de conhecimento:** com desenvolvedores e testadores trabalhando juntos, ao longo do tempo, um irá transferir o seu conhecimento para o outro, criando assim uma equipe multifuncional;
- **Documentação dinâmica:** algumas equipes ágeis afirmam que não documentam o sistema porque a manutenção destes artefatos é custosa. Usando os frameworks de BDD estes artefatos são gerados dinamicamente sem nenhum esforço adicional. Alguns, inclusive, geram relatórios em formato HTML, o que irá facilitar uma consulta posterior;

Por que comportamento?



- **Comunicação entre equipes:** na maioria das empresas de desenvolvimento de software é difícil fazer com que desenvolvedores e testadores trabalhem em conjunto para atingir um objetivo. **BDD possibilita esta integração porque os testadores podem escrever os cenários de testes para os desenvolvedores implementarem;**
- **Compartilhamento de conhecimento:** com desenvolvedores e testadores trabalhando juntos, ao longo do tempo, um irá transferir o seu conhecimento para o outro, criando assim uma equipe multifuncional;
- **Documentação dinâmica:** algumas equipes ágeis afirmam que não documentam o sistema porque a manutenção destes artefatos é custosa. Usando os frameworks de BDD estes artefatos são gerados dinamicamente sem nenhum esforço adicional. Alguns, inclusive, geram relatórios em formato HTML, o que irá facilitar uma consulta posterior;



Vamos praticar?





OXETECH Academy



SECTI
Secretaria de Estado de
Tecnologia e Inovação
e Desenvolvimento



 Softex

REGISTERED BY
AGÊNCIA REGULADORA
DE PROTEÇÃO

CONSUMO FEDERAL
BRASIL
Ministério da Economia